

Local Ollama Benchmark Update for LLM vs. RLM

Abstract

This update records a local replication of the LLM versus Recursive Language Model (RLM) benchmark suite using an on-device Ollama deployment instead of the earlier hosted-model runs. All results in this document come exclusively from a completed local batch executed with `qwen2.5:7b` through Ollama's OpenAI-compatible endpoint. The benchmark consisted of five paired LLM and RLM runs for each of seven tasks: Battle of the Bastards, AuthProxy long-context retrieval, Clinical long-context extraction, Launch Note App, PDF Cersei Warning, an under-specified TSP, and a stochastic adaptive TSP. We report latency and token usage from the raw run logs and note one failed sample in the stochastic adaptive TSP RLM condition. The main result is that RLM incurred substantially higher latency and token cost than the paired LLM baseline on every task in this local setting, while also exhibiting one run failure on the most computationally demanding benchmark.

1. Experimental Setup

1.1 Local Inference Configuration

- Workspace: `/Users/abhigyansekhar/Desktop/RLM-FULL`
- Backend: `ollama`
- Model: `qwen2.5:7b`
- Interface: OpenAI-compatible local endpoint at `http://127.0.0.1:11434/v1`
- Run count: 5 paired runs per benchmark
- Batch artifacts: `/Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama`

1.2 Benchmarks

The local batch covered these seven benchmark pairs:

1. Battle of the Bastards
2. AuthProxy Long Context
3. Clinical Long Context
4. Launch Note App
5. PDF Cersei Warning
6. Under-specified TSP
7. Stochastic Adaptive TSP

Each benchmark was executed as a matched pair:

- LLM: baseline single-pass model call
- RLM: recursive execution with tool/code iterations under the local environment

1.3 Data Source

This update uses only the local Ollama batch artifacts and excludes prior Gemini-based notes. The supporting files are:

- Raw per-run logs: `/Users/abhigyanshekhhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-o`
- Batch summary: `/Users/abhigyanshekhhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-o`
- Raw extracted table: `/Users/abhigyanshekhhar/Desktop/RLM-FULL/benchmark_runs/20260403-12115`
- Aggregate extracted table: `/Users/abhigyanshekhhar/Desktop/RLM-FULL/benchmark_runs/20260403-`

2. Results

2.1 Aggregate Performance

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Battle of the Bas-tards	LLM	5/5 (100%)	7.803	7.756	275.0	38.6	313.6
Battle of the Bas-tards	RLM	5/5 (100%)	108.379	73.517	9145.0	440.0	9585.0
AuthProxy Long Con-text	LLM	5/5 (100%)	23.689	23.109	1082.0	166.0	1248.0
AuthProxy Long Con-text	RLM	5/5 (100%)	179.107	173.042	11749.0	1251.6	13000.6
Clinical Long Con-text	LLM	5/5 (100%)	104.282	111.993	1645.0	1179.6	2824.6
Clinical Long Con-text	RLM	5/5 (100%)	137.302	52.487	11503.8	1017.2	12521.0
Launch Note App	LLM	5/5 (100%)	91.281	81.166	257.0	1044.6	1301.6

Benchmark	Mode	Success	Avg Wall (s)	Median Wall (s)	Avg Input Tokens	Avg Output Tokens	Avg Total Tokens
Launch Note App	RLM	5/5 (100%)	273.112	276.742	10956.8	2154.2	13111.0
PDF Cersei Warning	LLM	5/5 (100%)	132.849	133.707	4096.0	912.4	5008.4
PDF Cersei Warning	RLM	5/5 (100%)	212.396	191.413	19444.4	951.6	20396.0
Under-specified TSP	LLM	5/5 (100%)	108.994	106.786	209.0	1115.6	1324.6
Under-specified TSP	RLM	5/5 (100%)	510.680	369.738	20109.2	3406.0	23515.2
Stochastic Adaptive TSP	LLM	5/5 (100%)	164.309	185.630	318.0	1052.6	1370.6
Stochastic Adaptive TSP	RLM	4/5 (80%)	843.710	700.832	24390.0	3300.5	27690.5

2.2 Key Observations

1. RLM was slower than LLM on every benchmark in this local **qwen2.5:7b** setting.
2. RLM consumed substantially more tokens than LLM on every benchmark, typically by an order of magnitude.
3. The largest local cost gap appeared on the two TSP-style reasoning tasks:
 - Under-specified TSP: RLM averaged **510.680s** and **23515.2** tokens vs LLM at **108.994s** and **1324.6** tokens.
 - Stochastic Adaptive TSP: RLM averaged **843.710s** and **27690.5** tokens over successful runs vs LLM at **164.309s** and **1370.6** tokens.
4. The local run set included one failed RLM sample:
 - **stochastic_tsp** run 004 exited with code **-9**
 - this failed run was retained in the raw table and excluded from averaged RLM metrics for that benchmark

3. Interpretation

3.1 Cost of Recursive Execution

The local replication makes the overhead of recursive execution explicit. Even on relatively short tasks such as Battle of the Bastards, RLM averaged 108.379s compared with 7.803s for the baseline LLM. This suggests that on-device recursion, code execution, and iterative tool use introduce substantial latency overhead when the underlying local model is modest in capability and speed.

3.2 Long-Context Behavior

For long-context tasks such as AuthProxy, Clinical, and PDF Cersei Warning, the token gap was especially pronounced. For example:

- AuthProxy: 1248.0 average total tokens for LLM vs 13000.6 for RLM
- PDF Cersei Warning: 5008.4 average total tokens for LLM vs 20396.0 for RLM

This indicates that recursive decomposition in the local setup increases context processing cost significantly.

3.3 Reliability on Hard Reasoning Tasks

The stochastic adaptive TSP benchmark was the least stable local task for RLM, with one failed run out of five. Combined with the very high average wall-clock time, this suggests that the local `qwen2.5:7b` configuration is stressed most heavily by multi-step exact reasoning benchmarks under recursion.

4. Threats to Validity

1. These findings are specific to a local Ollama deployment using `qwen2.5:7b`.
2. They should not be conflated with earlier hosted-model results or Gemini-based runs.
3. The present document emphasizes latency and token usage because those metrics were reliably available from every raw run log.
4. Accuracy has to be interpreted by reading the individual run logs; this update does not yet include a second-pass human scoring layer or inter-rater agreement.
5. The sample size is five runs per task, which is useful for local profiling but still limited for strong statistical claims.

5. Conclusion

This local replication demonstrates that, on `qwen2.5:7b` via Ollama, RLM is consistently more expensive than the paired LLM baseline in both wall-clock latency and token usage across all tested benchmarks. The difference is moderate

on some retrieval-style tasks and very large on recursive reasoning tasks such as the TSP benchmarks. The local batch also exposed one failed RLM run on the stochastic adaptive TSP task. These results are best interpreted as a local systems-profile of LLM vs RLM behavior under an on-device open-weight model rather than as a direct substitute for higher-powered hosted-model evaluations.

Appendix A. Raw Data Source

The complete per-run markdown logs and extracted CSV tables are stored under:

`/Users/abhigyansekhar/Desktop/RLM-FULL/benchmark_runs/20260403-121151-ollama-qwen2.5-7b`